BERZIET UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ENCS3340

Artificial intelligence

Project Report 1

Prepared By:

Layal Hajji 1220871

Basmallah Abu Hakema 1220184

Supervised by:
Dr. Yazan Abu Farha

Section:
Layal Hajji 1
Basmala Abu Hakema 4

Problem Formulation

Initial State

The initial state is a random assignment of packages to vehicles, generated while respecting each vehicle's weight capacity. This state is built using the generate_initial_state() function, which attempts to allocate each package to a vehicle without exceeding its capacity.

Starting Point

The shop or depot located at coordinate (0, 0) is the fixed starting point for all delivery routes.

Actions (Operators / Successor Functions)

Actions represent modifications to the current assignment of packages to vehicles. Examples include:

- Moving a package from one vehicle to another (if capacity allows),
- Shuffling the order of packages in a vehicle's route.

The successor function get_random_neighbor() generates a new state by making a small, random change to the current state (e.g., reassigning one package).

State Space

The state space includes all valid combinations of package-to-vehicle assignments where:

- No vehicle exceeds its weight capacity,
- Each package is assigned to at most one vehicle.

The state also implicitly includes the route order for deliveries by each vehicle.

Goal State

A goal state is an assignment where:

- All packages are assigned
- The total delivery cost (distance + priority penalties + capacity violations) is minimized.

Goal Test

A goal test checks whether:

- The current state satisfies all package assignments without violating vehicle capacities,
- The cost function has reached a minimal or sufficiently low value (e.g., cost stabilizes or is lower than previous states).

Path

A path is a sequence of valid state transitions (package reallocations) from the initial state toward a better solution.

For Simulated Annealing, the path evolves through probabilistic state changes that occasionally accept worse solutions to escape local minima.
For the Genetic Algorithm, the path is the evolutionary trajectory from a random population to the best-fit individual through selection, crossover, and mutation.

Path Cost

The cost of a path is determined by the compute_cost() or fitness() functions and includes:

- Total travel distance for each vehicle,
- Penalties based on package priority and delivery order,
- Penalties for exceeding vehicle capacities or leaving packages unassigned.

Solution

A solution is a final package-to-vehicle assignment that minimizes delivery cost. It is obtained either:

- From Simulated Annealing when temperature drops below a threshold or max iterations are reached,
- From the Genetic Algorithm after completing the specified number of generations.

Algorithm Descriptions and Heuristics

A. Simulated Annealing

Initial Solution Generation: The algorithm begins with a randomly generated, feasible assignment of packages to vehicles. Basic capacity constraints are respected to avoid starting from an invalid state.

Neighbor Solution Generation: Neighboring solutions are created by randomly swapping packages between two vehicles or altering the delivery order within a single vehicle's route.

Priority Heuristic: To handle package priority, the cost function adds a penalty for high-priority packages that are scheduled late. This encourages early delivery of important packages.

Cooling Schedule: An exponential cooling schedule is used, updating the temperature with $T_new = \alpha * T_old$, where α is set to 0.95. The algorithm stops when the temperature drops below a threshold.

B. Genetic Algorithm

Chromosome Encoding: Each chromosome represents a solution as a dictionary, where each key is a vehicle ID and the value is an ordered list of assigned package IDs.

Fitness Function: The fitness function calculates the total delivery distance and applies penalties for constraint violations (e.g., capacity, unassigned packages) and late deliveries of high-priority items.

Selection Method: Tournament selection is used to choose parents. It randomly selects a few individuals and chooses the best among them based on fitness, promoting good solutions.

Crossover and Mutation: Uniform crossover exchanges package assignments between parent chromosomes. Mutations involve swapping packages between routes or changing the delivery order within a vehicle to introduce diversity.

Heuristics for Improvement: To improve initial population quality, clustering is used to assign nearby packages to the same vehicle. Mutations are also biased toward improving priority handling or reducing travel distance.

Effects of Parameter Tuning

In the Simulated Annealing (SA) algorithm, we tuned the cooling rate, a parameter that dictates how quickly the temperature decreases during the optimization process. We tested values of 0.90, 0.95, and 0.99. A lower cooling rate like 0.90 led to faster convergence but resulted in suboptimal solutions due to limited exploration; it often got trapped in local minima. Increasing the cooling rate to 0.95 improved results slightly by allowing more exploration before settling. The best performance was observed with a cooling rate of 0.99, which allowed the algorithm to explore the solution space more thoroughly and avoid premature convergence, albeit at the cost of longer execution time. This demonstrates that a slower cooling schedule leads to higher solution quality, especially for complex search spaces.

For the Genetic Algorithm (GA), we focused on tuning the mutation rate, which controls the probability of random changes in individuals. We evaluated mutation rates of 0.01, 0.05, and 0.10. A low mutation rate (0.01) caused the population to converge quickly but resulted in poor solution quality due to lack of diversity. On the other hand, a higher mutation rate (0.10) maintained diversity across generations, preventing premature convergence and enabling the algorithm to explore a broader range of solutions. This setting produced the best final results, though convergence was slower. The moderate value (0.05) offered a balance between exploration and stability. Overall, a higher mutation rate improves effectiveness and diversity at the cost of slower convergence, highlighting the importance of tuning for optimal performance.

Test cases

Test case 1:

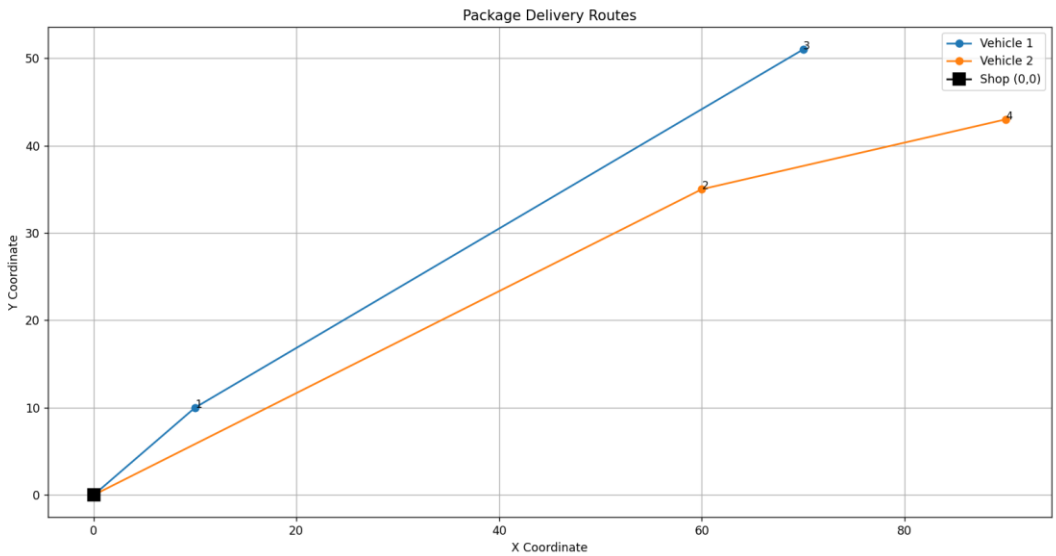
C: > Users > Layal > Desktop > BZU > Current Semester > AI > Project1 > packages.csv > data

```csv
id,x,y,weight,priority
1,10,10,30,3
2,60,35,60,1
3,70,51,50,2
4,90,43,40,2
5,55,33,150,1
```

C: > Users > Layal > Deskto

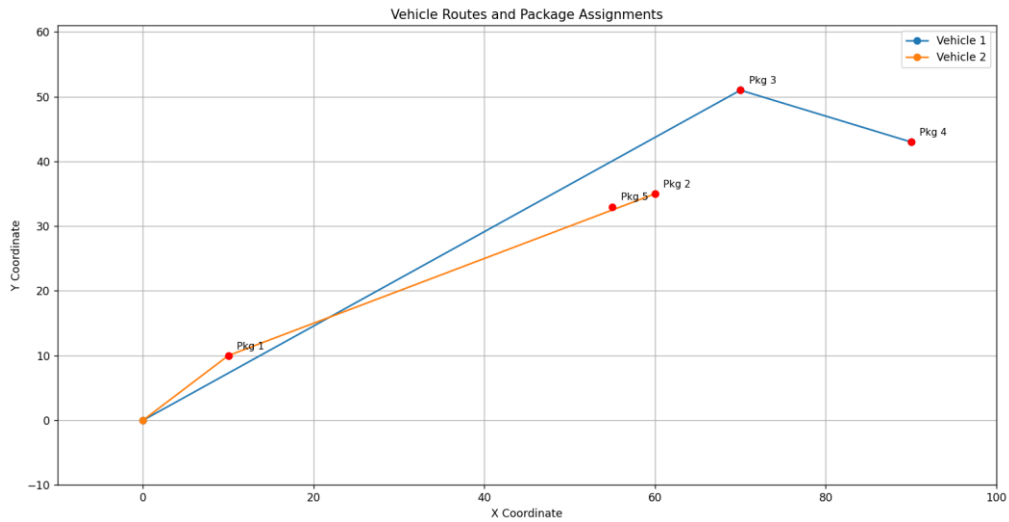
```csv
id,capacity
1,100.0
2,100.0
```

Simulated annealing




```
✅ Simulated Annealing completed.
Resulting Cost: 187.32319415959668
Resulting Assignment:
Vehicle 1: ['2', '4']
Vehicle 2: ['1', '3']
```

Genetic Algorithm




```
Best assignment of packages to vehicles (with some packages left out):
Package 1 -> Vehicle 2
Package 2 -> Vehicle 2
Package 3 -> Vehicle 1
Package 4 -> Vehicle 1
Package 5 is left out.
```

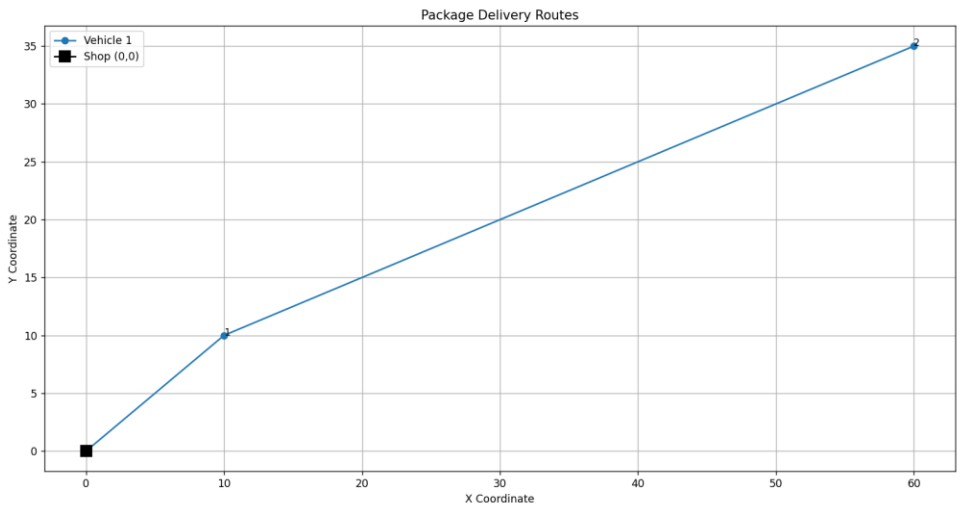
Test case 2:

Here the distance was chosen over the priority

```csv
id,x,y,weight,priority
1,10,10,50,3
2,60,35,50,1
3,70,51,50,2
```

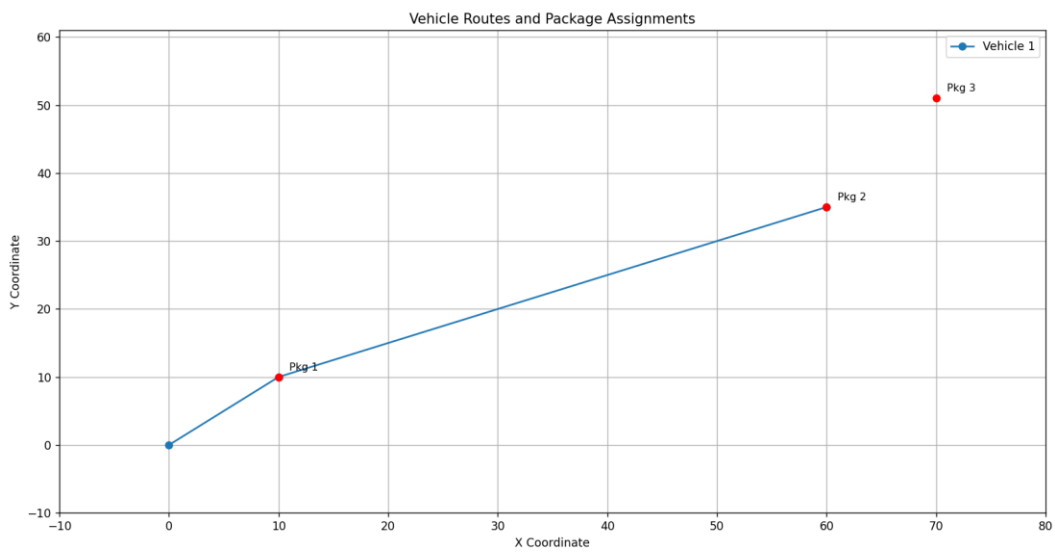
```csv
id,capacity
1,100.0
```

Simulated annealing




```
✅ Simulated Annealing completed.
Resulting Cost: 70.04383506122569
Resulting Assignment:
Vehicle 1: ['1', '2']
```

Genetic Algorithm




```
Gen 499: max=16.3279128885804 min=16.3279128885804 avg=16.33

Best assignment of packages to vehicles (with some packages left out):
Package 1 -> Vehicle 1
Package 2 -> Vehicle 1
Package 3 is left out.
```

Test case 3:

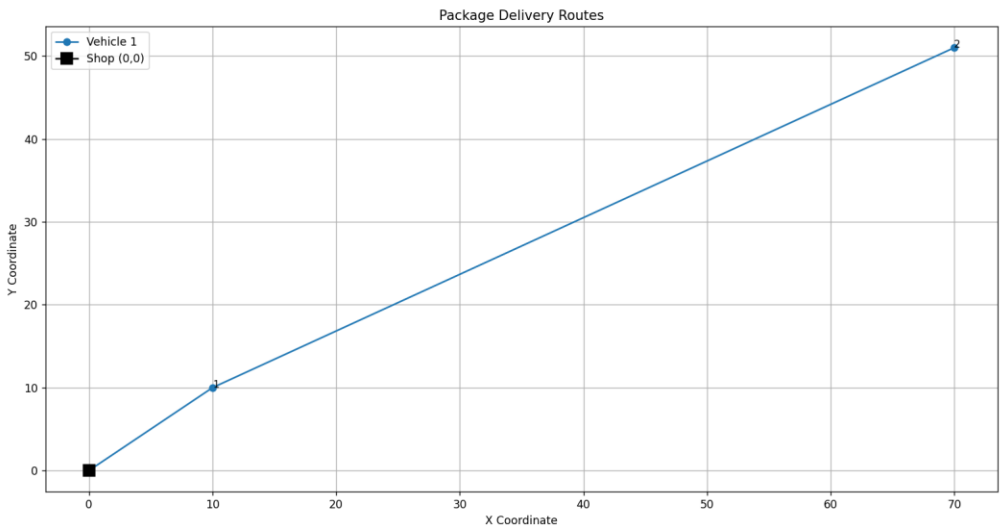
We switched the x,y coordinate of package 2 and 3

Here the priority was chosen over the distance

```
id,x,y,weight,priority
1,10,10,50,3
2,70,51,50,1
3,60,35,50,2
```

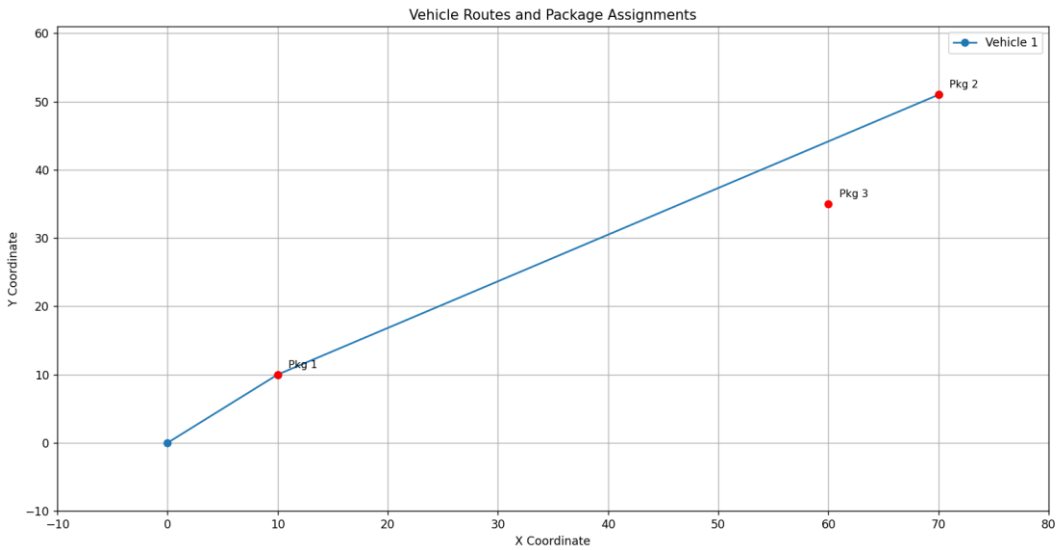
```
id,capacity
1,100.0
```

Simulated annealing




```
✅ Simulated Annealing completed.
Resulting Cost: 86.8126248198276
Resulting Assignment:
Vehicle 1: ['1', '2']
```

Genetic Algorithm



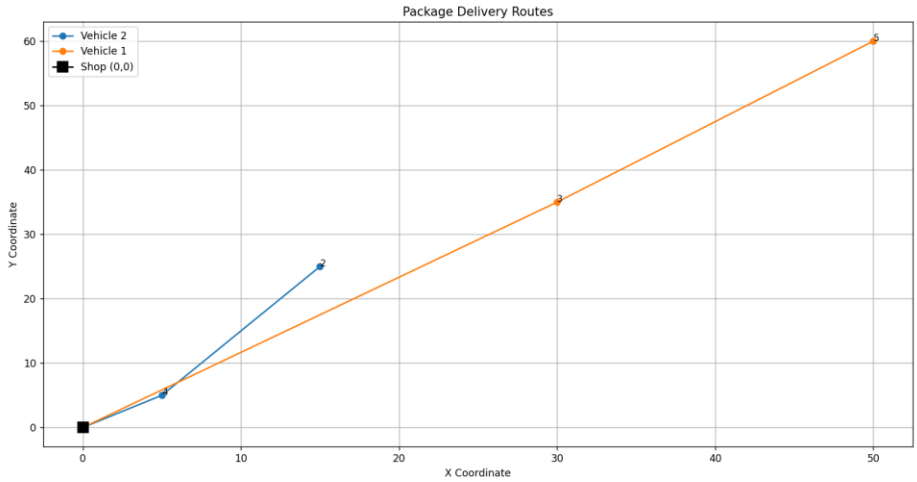

```
Gen 499: max=15.984991013751412 min=15.984991013751412 avg=15.98

Best assignment of packages to vehicles (with some packages left out):
Package 1 -> Vehicle 1
Package 2 -> Vehicle 1
Package 3 is left out.
```

Test case 4:

```
id,x,y,weight,priority
1,10,20,5,2
2,15,25,10,1
3,30,35,8,3
4,5,5,4,2
5,50,60,12,1
```

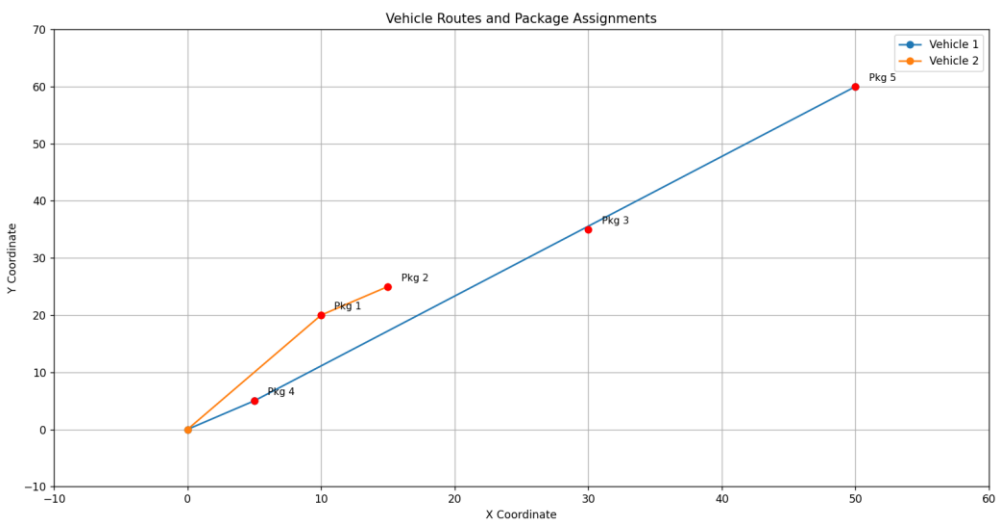
```
id,capacity
1,20
2,15
```

Simulated annealing




```
✅ Simulated Annealing completed.
Resulting Cost: 107.54509106049204
Resulting Assignment:
Vehicle 1: ['3', '5']
Vehicle 2: ['4', '2']
```

Genetic Algorithm




```
Gen 499: max=35.26621992359687 min=33.906315413048915 avg=35.25

Best assignment of packages to vehicles (with some packages left out):
Package 1 -> Vehicle 2
Package 2 -> Vehicle 2
Package 3 is left out.
Package 4 -> Vehicle 1
Package 5 -> Vehicle 1
```